Six Spur Ranch & Rescue

Website Developer Handoff Guide

sixspurranch.org | Prepared by Jay Lefler | October 2026

This guide gives the owner of Six Spur Ranch & Rescue, and any web developer they hire, what they need to take over, run and maintain sixspurranch.org without help from the original developer. It covers who owns what, how the system is built, every major feature, how changes are deployed, the known pitfalls, and a step-by-step checklist for transferring control.

No passwords, API secrets or tokens are written in this document. Section 7 explains where each secret lives and how it is handed over and rotated.

A companion file, `docs/HANDOFF_INVENTORY.md` in the code repository, lists every live AWS resource (functions, tables, routes, schedules, buckets and so on) as pulled directly from the AWS account. Read it alongside this guide.

Section	Contents
1	Project status
2	Accounts, ownership and what must be transferred
3	Architecture overview
4	Infrastructure identifiers
5	Feature inventory
6	Third-party integrations and scheduled jobs
7	Credentials and secrets
8	Deploying changes, and known pitfalls
9	Handoff checklist
10	First steps for a new developer

1. Project status

The site is feature-complete and in production. All major planned work was finished and verified, including live PayPal payments, refund automation, the shop checkout, SMS alerts (out of the AWS sandbox), the mobile UI audit and the deployment-script fixes. There is no unfinished feature work in progress.

One item to verify: a real $1/month PayPal recurring donation was left active on purpose to confirm that automatic monthly charges fire (first expected around September 17, 2026). Check that it charged and recorded in donation history, then cancel it in PayPal.

Second item to verify: the Facebook Page token (stored in the `sixspur/meta-api` secret) was due to expire around early October 2026. The `sixspur-refresh-facebook-tokens` Lambda is scheduled to renew it automatically every 30 days and was confirmed working, but check its recent CloudWatch logs and post one test item to Facebook to confirm the renewal actually happened.

Full build history, including why things were built the way they were and every bug found along the way, is in `docs/SixSpur_Project_Notes.md` in the repo.

2. Accounts, ownership and what must be transferred

Everything the site depends on is listed below. For each one, Six Spur should end up as the owner (or at least a full administrator), and the original developer's access should be removed once the transfer is confirmed.

Asset	Identifier	Transfer action
Source code (GitHub)	Repo **UnboxingTreasures/sixspur-website**	Transfer the repo to a GitHub account or organization Six Spur controls (Settings → Transfer ownership). Amplify must then be reconnected to the repo at its new location.
AWS account	Account 658965339779, region us-east-1	Root user email/MFA and billing must belong to Six Spur. Create IAM users for Six Spur and any new developer, then delete the original developer's IAM user and access keys.
Hosting (AWS Amplify)	App ID d3u4k69gjq52xt	Reconnect to the transferred GitHub repo (App settings → Repository). Verify a build runs.
Domain	sixspurranch.org (sixspurranch.net redirects to it)	Confirm the registrar account and renewal payment are in Six Spur's name. Check DNS records (see inventory) and auto-renew.
PayPal business account	Live webhook 06U07907R9459342C; 3 subscription plans	Already Six Spur's account. Make sure the developer API app credentials are rotated after handoff (Section 7).
Meta / Facebook	App "Six Spur Social Posting", App ID 1056766383539468	Make Six Spur's Facebook account an Admin of the app in developers.facebook.com, then remove the original developer. The Page and Instagram account are Six Spur's.
Site admin logins	Super-admins: sixspurrescue@gmail.com, jaylefler1974@gmail.com	These two emails are **hard-coded** in `lambda/adminUserAccess` (server side) and in the admin User Access page (client side). Removing jaylefler1974@gmail.com takes a small code change and redeploy, then removing that user from Cognito.

Asset	Identifier	Transfer action
SMS alert recipients	Managed in Admin → User Access	Jay's phone currently receives every staff alert (donations, orders, applications, inbound email). Remove it from the SMS recipients list, and check that no older environment variable (`SMS_RECIPIENTS`, `RICHARD_PHONE_NUMBER`) on the adoptionApplication or contactForm Lambdas still holds it.
Email	sixspurrescue@gmail.com	Six Spur's account. Used for admin login, PayPal, notifications and contact.
Social accounts	Instagram @sixspurrescue, TikTok @sixspur, YouTube @SixSpurRanchandRescue	Six Spur's own accounts; no transfer needed.

3. Architecture overview

The site is a serverless application on AWS. There are no servers to patch; costs scale with use.

Layer	Technology	Notes
Front end	Next.js (React)	Built and hosted by AWS Amplify. Every push to the connected Git branch triggers a build and deploy.
API	Amazon API Gateway (HTTP API)	ID vvabeaemg5. Routes call Lambda functions. Protected routes use a JWT authorizer (gyxmij) backed by Cognito.
Back-end logic	AWS Lambda (Node.js)	One function per job: donations, refunds, shop, orders, blog, mailing list, social posting, notifications, admin. Full list in the inventory.
Data	Amazon DynamoDB	Donors, donations, orders, products, subscribers, sms_recipients and others. Table list and keys in the inventory.
Files/images	Amazon S3 + CloudFront	Uploaded images and static assets, served through the CDN.
User accounts	Amazon Cognito	Donor sign-up, email verification, login, password reset. Admins are Cognito users flagged as admin.
Payments	PayPal (live)	One-time and recurring donations, shop checkout, refunds. PayPal calls back into the site via a webhook.
Messaging	AWS SMS/SES/SNS	SMS alerts to staff and transactional emails (receipts, verifications, refund confirmations).
Scheduling	Amazon EventBridge	Runs the Facebook token refresh every 30 days.

The codebase follows the same patterns as Jay's Unboxing Treasures platform, which served as the reference implementation. A developer familiar with Next.js and AWS serverless will recognize the structure.

4. Infrastructure identifiers

Item	Value
AWS account	658965339779
Region	us-east-1
AWS CLI profile name used in scripts	sixspur

Item	Value
Amplify app ID	d3u4k69gjq52xt
API Gateway ID	vvabeaemg5
JWT authorizer ID	gyxmij
GitHub repo	UnboxingTreasures/sixspur-website (to be transferred)
Domain	sixspurranch.org; sixspurranch.net redirects
Organization EIN (appears on donation receipts)	41-4123317
PayPal live webhook ID	06U07907R9459342C
PayPal plan: $5/month	P-4BG26895CH4480936NKBVYXQ
PayPal plan: $10/month	P-78526558P2276734XNKBVYXQ
PayPal plan: $20/month	P-7AW61509S2085861PNKBVYXY
Facebook app ID	1056766383539468

Note: deploy scripts in the repo reference the AWS CLI profile name `sixspur`. A new developer should either create a local profile with that name or update the scripts.

5. Feature inventory

Donations

- One-time and recurring ($5/$10/$20 monthly) PayPal donations, live and verified end to end.
- Donor accounts: sign-up with email verification, login, password reset, donation history with year/month filter, downloadable receipts.
- Refund automation from the admin panel: full and partial refunds, safe to retry (idempotent), with email confirmations.

Fundraisers

- Fundraiser campaigns shown as animated SVG thermometers, with color that heats up as the goal approaches and fireworks at 100%.
- Archive lifecycle: finished fundraisers move to an archive.

Shop

- Product catalog with admin-editable variants (e.g. size × color), with stock tracked per combination.
- Cart stored in the browser; stock is reserved atomically in DynamoDB at checkout so items cannot oversell.
- PayPal checkout, customer order history, admin Orders page.
- Shop settings admin: shipping rate and return/mailing address.

Content

- Blog/news with admin create/edit/delete; comments that anyone can read, logged-in donors can write, with one level of replies and admin delete.

- Recently adopted animals, wish list page, friendly empty states for news and shop.

Communication

- Mailing list admin panel; the donor opt-in checkbox and the subscribers list stay in sync both ways.
- Post to Facebook and Instagram from the admin panel.
- SMS alerts to staff for events like new donations and orders. Recipients are managed in the admin panel (stored in the sms_recipients table and read fresh each time an alert is sent).

Admin and security

- All ~11 admin functions require a valid Cognito login token plus an admin flag (the older Basic Auth was fully removed).
- Page-transition loading bar and a full mobile UI pass.

6. Third-party integrations and scheduled jobs

PayPal

- Runs against Six Spur's live PayPal business account using a REST API app (client ID + secret).
- The live webhook (ID above) tells the site about completed payments, subscription renewals and refunds. If the site's domain or API URL ever changes, the webhook URL in the PayPal developer dashboard must be updated.
- Subscription plan IDs above are referenced by the site; creating new plan amounts means creating new plans in PayPal and updating the site's configuration.

Facebook / Instagram

- Posting uses a long-lived Page access token stored in AWS.
- The token lives in the `sixspur/meta-api` secret. A Lambda (`sixspur-refresh-facebook-tokens`) runs every 30 days on an EventBridge schedule to renew it. If social posting stops working, check this function's logs in CloudWatch first.
- If the Facebook app admin changes or the Page password is reset, a new token may need to be generated once by hand, after which the schedule keeps it fresh.

SMS and email

- SMS has been moved out of the AWS sandbox, so it can text any number. AWS bills per message, with a $50/month spend limit on the account.
- Staff get texts for four events: donations, shop orders, adoption applications and inbound email replies.
- Transactional emails (verification, receipts, refunds, shipping notices) are sent from noreply@sixspurranch.org, which cannot receive mail; emails point people to the website contact form instead.
- Replies to contact-form conversations are received and processed by the `processIncomingEmail` Lambda, which strips quoted text and phone signatures before saving.
- PayPal secret: `sixspur/paypal-api` in Secrets Manager. It must hold the credentials of the app in Six Spur's own PayPal dashboard. An older, unused PayPal app from early in the project exists and is not referenced anywhere.

7. Credentials and secrets

Secrets are stored in AWS (Lambda environment variables, Amplify environment variables and/or AWS Secrets Manager / Parameter Store). The inventory file lists the *names* of every environment variable and secret, never the values. To hand over control:

- ■ Six Spur owner signs in to the AWS root account and confirms they control its email, password and MFA device.
- ■ Create a new IAM user (or IAM Identity Center user) for the new developer. Do not share one login.
- ■ **Rotate after handoff:** generate a new PayPal REST app secret and update it in AWS; regenerate the Facebook Page token; delete the original developer's AWS access keys.
- ■ Change the password on sixspurrescue@gmail.com if it was ever shared during the build, and turn on 2-step verification.
- ■ Never paste secrets into email, chat or documents. Update them only inside AWS or with the AWS CLI on the developer's own machine.

Important when updating any Lambda's environment variables: AWS replaces the *entire* set when you save. Always fetch the current variables, change only what you need, and write the whole set back (see Section 8).

8. Deploying changes, and known pitfalls

Front end

Commit and push to the connected branch. Amplify builds and deploys automatically; watch the build in the Amplify console. A local build is not required.

```
git add -A
git commit -m "Describe the change"
git push
```

Lambda functions

The repo contains deploy scripts that zip each function and upload it with the AWS CLI using the `sixspur` profile. Read the script before running it. Lessons learned the hard way:

- **Always include adminAuth.js in the zip for admin functions.** Leaving it out causes the function to crash on every request.
- **Never overwrite environment variables wholesale.** Use a fetch-merge-patch pattern: read the current variables, merge the change, write them all back. Overwriting has previously wiped live secrets.
- **deploy-social.sh** is safe to re-run: it will not duplicate API Gateway routes and will not overwrite the live Facebook token.
- **Use JSON for --environment, never the CLI shorthand.** The shorthand silently breaks on any value containing a comma (such as a list of phone numbers).
- **Before running a deploy.sh that hasn't been run in a while,** compare its zip file list with what's in the folder. Stale lists have broken live pages before.
- **A new Lambda route must also be registered in API Gateway** (`aws apigatewayv2 create-route`). If it isn't, the browser reports "Failed to fetch" because the CORS preflight has nowhere to go.
- IAM policy files in the repo have drifted from the live policies before. When a function gets an access-denied error, check the live role in IAM, not just the repo copy.

- Facebook photo posts return their ID as `post_id`; the code falls back to `id`. Keep that if the posting code is modified.

Example of a safe environment-variable change (adds or updates one variable, keeps the rest):

```
FN=functionName
aws lambda get-function-configuration --function-name $FN --profile sixspur \
--query 'Environment.Variables' > vars.json
# edit vars.json locally, then:
aws lambda update-function-configuration --function-name $FN --profile sixspur \
--environment "Variables=$(cat vars.json)"
rm vars.json
```

Monitoring

- Each Lambda writes logs to CloudWatch (log group `/aws/lambda/<name>`). Start there for any error.
- Set up an AWS Budget alert on the account so unexpected charges are noticed.
- PayPal's developer dashboard shows webhook delivery history; failed deliveries point to API issues.

9. Handoff checklist

Original developer provides

- ■ This guide (PDF).
- ■ docs/HANDOFF_INVENTORY.md and the project notes file, committed to the repo.
- ■ Transfer of the GitHub repo to Six Spur's GitHub account.
- ■ Confirmation of which AWS, domain and Facebook developer accounts are in whose name, and help moving any that are not yet Six Spur's.
- ■ Verification and cancellation of the $1/month test subscription.

Six Spur completes

- ■ Accept the GitHub repo transfer; reconnect Amplify to it and confirm a successful build.
- ■ Confirm control of the AWS root account and billing; add an AWS Budget alert.
- ■ Confirm domain registrar ownership and auto-renew.
- ■ Become sole admin of the Facebook developer app.
- ■ Create IAM access for the new developer.
- ■ Rotate secrets (Section 7).
- ■ Remove the original developer's phone from SMS recipients.
- ■ Remove the original developer's super-admin email from lambda/adminUserAccess and the admin page (code change + deploy), then from Cognito.
- ■ Remove the original developer's AWS IAM user/keys, GitHub access and Facebook app role.
- ■ Confirm the Facebook token renewed (Section 1).
- ■ Written acknowledgment that the handoff is complete, kept by both sides.

10. First steps for a new developer

- Read this guide, docs/HANDOFF_INVENTORY.md and the project notes in the repo.
- Clone the repo, install dependencies (`npm install`), and configure an AWS CLI profile named `sixspur` with your IAM credentials in us-east-1.
- Open the Amplify console and confirm the latest build and the environment variables it uses.
- Walk through the live site as a donor and as an admin (create a test account; avoid live payments unless intended).
- Review CloudWatch logs for recent errors and confirm the refreshFacebookToken schedule ran in the last 30 days.
- Before any Lambda change, read Section 8 and the relevant deploy script.